Технічне Завдання (ТЗ) та Єдиний Стандарт Модульної Системи

Проєкт: Модульний конвеєр комп'ютерного зору ("Комбайн жестів та дій")

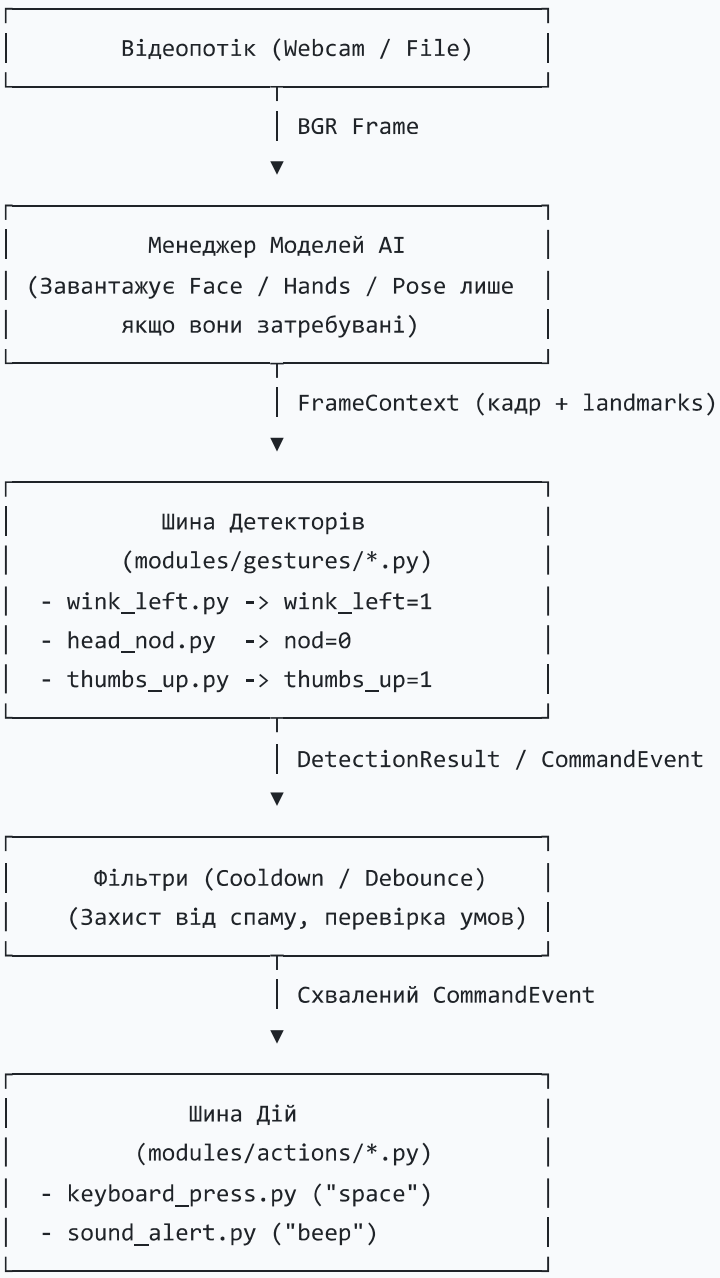
1. Загальна концепція та філософія

Мета системи — створити гнучкий, розширюваний та легковажний конвеєр обробки комп'ютерного зору за принципом **Plug-and-Play (підключи і працюй)**.

Ключові принципи:

1. **Файлова модульність**: Будь-який жест або дія — це ізольований `.py` файл зі стандартизованим інтерфейсом.
2. **Нульове дублювання та розумне ядро (On-Demand Models)**: Ядро саме аналізує підключені файли і завантажує в пам'ять лише ті AI-моделі, які реально потрібні. Якщо підключено 3 модулі обличчя — модель обличчя проганяється один раз на кадр для всіх трьох.
3. **Відокремлення розпізнавання від реакції**: Жест лише фіксує факт події (повертає значення/сигнал). Що робити з цим сигналом (натиснути клавішу, видати звук, відправити HTTP-запит) — вирішує модуль дії.
4. **Конфігурабельність без зміни коду**: Зв'язування жестів з діями та налаштування порогів чутливості виконується через простий декларативний конфіг (`pipeline.yaml`).

2. Архітектура системи та потоки даних



3. Єдиний стандарт структур даних (Data Transfer Objects)

Кожен компонент спілкується через суворо типізовані структури даних.

3.1. `FrameContext` (Контекст кадру, що надходить у детектори)

```
python
from dataclasses import dataclass, field
from typing import Optional, Dict, Any
import numpy as np

@dataclass
class FrameContext:
    frame: np.ndarray             # Сирий кадр OpenCV (BGR)
    frame_rgb: np.ndarray         # Кадр у форматі RGB (для моделей)
    timestamp: float              # Поточний час (time.time())
    frame_id: int                 # Порядковий номер кадру
    width: int                    # Ширина кадру в пікселях
    height: int                   # Висота кадру в пікселях

    # Словник з результатами інференсу моделей (заповнюється лише якщо модель завантажена)
    landmarks: Dict[str, Any] = field(default_factory=dict)
    # Приклади ключів:
    # "face": список точок обличчя (468/478 точок або Blendshapes)
    # "hands": список знайдених рук (landmarks для правої/лівої руки)
    # "pose": 33 ключові точки тіла
```

3.2. `DetectionResult` (Результат роботи модуля-детектора)

```
python
@dataclass
class DetectionResult:
    triggered: bool               # Чи активний жест прямо зараз (True / False)
    value: float = 0.0            # Числове значення (0.0 .. 1.0 або дискретне 1.0 / 0.0)
    confidence: float = 1.0       # Впевненість детектора (0.0 .. 1.0)
    extra_data: Dict[str, Any] = field(default_factory=dict)
    # Наприклад extra_data={"angle": 18.5, "direction": "left"}
```

3.3. `CommandEvent` (Команда, що передається у модуль дії)

```
python
@dataclass
class CommandEvent:
    event_id: str                 # Унікальний ідентифікатор події (UUID або назва)
    gesture_name: str             # Назва жесту, що згенерував подію
    timestamp: float              # Час виникнення
    value: float                  # Передане значення
    params: Dict[str, Any]        # Налаштування зв'язки з pipeline.yaml
```

4. Специфікація модуля жесту (`modules/gestures/*.py`)

Кожен файл жесту повинен відповідати наступному контракту.

Обов'язкові змінні та методи:

1. `MODULE_NAME: str` — унікальне ім'я модуля.
2. `REQUIRES: list[str]` — список необхідних моделей (`"face"`, `"hands"`, `"pose"`).
3. `DEFAULT_CONFIG: dict` — параметри чутливості за замовчуванням.
4. Клас `GestureDetector`:
    - `__init__(self, config: dict)` — ініціалізація внутрішнього стану (історія кадрів, фільтри тощо).
    - `process(self, ctx: FrameContext) -> DetectionResult` — обробка кадру.

Еталонний шаблон: `modules/gestures/wink_left.py`

```
python
"""
Модуль детекції підморгування лівим оком.
"""
from typing import Dict, Any
from core.models import FrameContext, DetectionResult

MODULE_NAME = "wink_left"
REQUIRES = ["face"]

DEFAULT_CONFIG = {
    "eye_aspect_ratio_threshold": 0.18,  # Поріг закриття ока (EAR)
    "opposite_eye_open_threshold": 0.22, # Поріг відкритості другого ока
}

class GestureDetector:
    def __init__(self, config: Dict[str, Any]):
        self.config = {**DEFAULT_CONFIG, **config}

    def process(self, ctx: FrameContext) -> DetectionResult:
        face_data = ctx.landmarks.get("face")
        if face_data is None:
            return DetectionResult(triggered=False, value=0.0)

        # Отримуємо точки лівого та правого ока
        left_ear = self._calculate_ear(face_data, is_left=True)
        right_ear = self._calculate_ear(face_data, is_left=False)

        # Перевірка умови: ліве заплющене, праве відкрите
        is_winking = (
            left_ear < self.config["eye_aspect_ratio_threshold"] and
            right_ear >= self.config["opposite_eye_open_threshold"]
        )

        return DetectionResult(
            triggered=is_winking,
            value=1.0 if is_winking else 0.0,
            confidence=0.95,
            extra_data={"left_ear": left_ear, "right_ear": right_ear}
        )

    def _calculate_ear(self, face_landmarks, is_left: bool) -> float:
        # Розрахунок Eye Aspect Ratio за точками сітки обличчя
        return 0.25 # Заглушка обчислення
```

5. Специфікація модуля дії (`modules/actions/*.py`)

Модуль дії приймає сигнал і виконує операцію (клавіатура, миша, системна команда тощо).

Обов'язкові змінні та методи:

1. `MODULE_NAME: str` — унікальне ім'я модуля дії.
2. `DEFAULT_CONFIG: dict` — параметри за замовчуванням (наприклад, клавіша, час утримання).
3. Клас `ActionExecutor`:
    - `__init__(self, config: dict)` — ініціалізація (драйвери вводу, звуковий пристрій тощо).
    - `execute(self, event: CommandEvent) -> bool` — виконання дії. Повертає `True` у разі успіху.

Еталонний шаблон: `modules/actions/keyboard_press.py`

```
python
"""
Модуль емуляції натискання клавіші клавіатури.
"""
from typing import Dict, Any
from core.models import CommandEvent

MODULE_NAME = "keyboard_press"
DEFAULT_CONFIG = {
    "key": "space",        # Клавіша для натискання
    "mode": "tap"          # "tap" (одноразово), "down" (затиснути), "up" (відпустити)
}

class ActionExecutor:
    def __init__(self, config: Dict[str, Any]):
        self.config = {**DEFAULT_CONFIG, **config}
        # Ініціалізація бібліотеки емуляції клавіатури

    def execute(self, event: CommandEvent) -> bool:
        target_key = event.params.get("key", self.config["key"])
        # Емуляція натискання клавіші
        print(f"[ACTION: keyboard_press] Натиснуто клавішу: {target_key}")
        return True
```

6. Механізм обробки подій та фільтрації (Logic & Cooldown)

Щоб уникнути спаму (коли жест активний протягом 30 кадрів підряд і викликає 30 натискань клавіші за секунду), ядро містить стандартний обробник тригерів:

1. **Cooldown (Кулдаун)**: Мінімальний інтервал часу між повторними спрацьовуваннями (наприклад, не частіше разу на 0.8 с).
2. **Hold Time (Час утримання)**: Жест повинен залишатися активним безперервно протягом N N секунд для активації.
3. **Edge Triggering (Детекція фронту)**: Дія виконується лише в момент переходу стану з $0 \to 10 \to 1$ (Rising edge).

7. Формат конфігурації зв'язок (`pipeline.yaml`)

Користувач зв'язує компоненти в єдиний робочий пайплайн без написання коду:

```yaml
yaml
# pipeline.yaml
version: "1.0"

settings:
  camera_id: 0
  preview_window: true       # Показувати віконце налагодження (OpenCV)
  show_landmarks: true       # Малювати точки обличчя/рук
  fps_limit: 30

bindings:
  - id: "space_on_wink"
    name: "Пауза/плей через підморгування"
    gesture:
      file: "modules/gestures/wink_left.py"
      config:
        eye_aspect_ratio_threshold: 0.19
    filter:
      cooldown: 0.8          # Інтервал між спрацьовуваннями (сек)
      trigger_on: "rising"   # Спрацьовувати в момент заплющення
    action:
      file: "modules/actions/keyboard_press.py"
      config:
        key: "space"

  - id: "screenshot_on_peace"
    name: "Скріншот жестом миру"
    gesture:
      file: "modules/gestures/hand_peace.py"
      config:
        min_confidence: 0.8
    filter:
      cooldown: 2.0
    action:
      file: "modules/actions/system_command.py"
      config:
        command: "prntscrn"
```

8. Структура файлової системи проєкту

```
pose--/
│
├── SPECIFICATION.md          # Цей документ специфікації
├── pipeline.yaml             # Конфігурація підключених модулів
├── run.py                    # Головна точка входу (Комбайн-раннер)
│
├── core/                     # Ядро системи
│   ├── __init__.py
│   ├── models.py             # Датакласи (FrameContext, DetectionResult, CommandEvent)
│   ├── model_manager.py      # Розумне завантаження Face / Hands / Pose
│   ├── module_loader.py      # Динамічний імпорт модулів .py за шляхом
│   ├── pipeline.py           # Конвеєр виконання (кадр -> детектори -> фільтри -> дії)
│   └── visualizer.py         # Візуалізатор прев'ю з індикаторами спрацювання
│
├── modules/
│   ├── gestures/             # Модулі розпізнавання жестів
│   │   ├── __init__.py
│   │   ├── wink_left.py
│   │   ├── wink_right.py
│   │   ├── head_nod.py
│   │   ├── hand_thumbs_up.py
│   │   └── ...
│   │
│   └── actions/              # Модулі виконання дій
│       ├── __init__.py
│       ├── keyboard_press.py
│       ├── sound_alert.py
│       ├── mouse_click.py
│       └── ...
│
└── utils/                    # Допоміжна математика (EAR, вектори кутів)
    ├── __init__.py
    └── geometry.py
```

9. Етапи реалізації

1. **Етап 1: Базове ядро та контракти**: Створення структури `core/`, датакласів та динамічного завантажувача модулів.

2. **Етап 2: AI Model Manager**: Реалізація обгортки над MediaPipe (FaceMesh, Hands, Pose) з механізмом лінивого завантаження за запитом.
3. **Етап 3: Перший робочий жест та дія**: Реалізація `wink_left.py` (підморгування) + `keyboard_press.py` (натискання клавіші).
4. **Етап 4: Візуальний дебаг у вікні OpenCV**: Вивід FPS, накладання точок активних моделей та кольорові індикатори спрацювання жестів.
5. **Етап 5: Додавання нових жестів**: Кивки головою, повороти, жести руками (палець вгору, кулак).